

class 07 - Principal Component Analysis (PCA)

Omar Siddiqui (PID: A19093654)

Overview

PCA = **Principal Component Analysis** — reduces high-dimensional data into fewer dimensions while preserving the most important variation.

- Data with many variables is hard to visualize/interpret directly
- PCA finds new axes (PCs) that capture the *most* variance in the data
- PC1 captures the most variance, PC2 the second most, etc.
- Think of it like finding the “best angle” to look at your data from

Key idea: We’re not losing data, we’re *summarizing* it. PCA is unsupervised — no group labels needed.

Section 1: PCA of UK Food Data

Background

Dataset from UK DEFRA: consumption (g/person/week) of 17 food types across England, Wales, Scotland, and N. Ireland in 1997.

- 17 rows (foods), 4 columns (countries)
- Countries = observations, foods = variables
- Equivalent to samples vs. genes in RNA-seq

Data Import

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url, row.names=1)
```

Q1: How many rows and columns are in your new data frame named x?
What R functions could you use to answer this?

```
dim(x)
```

```
[1] 17  4
```

```
nrow(x)
```

```
[1] 17
```

```
ncol(x)
```

```
[1] 4
```

17 rows (foods) and 4 columns (countries). You can use `dim()`, `nrow()`, or `ncol()` to check this.

Checking Your Data

Notes: - Always inspect imported data before doing anything with it - `head()` shows the first 6 rows by default - **Never leave `View()` uncommented in your Quarto doc** — it will break rendering

```
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

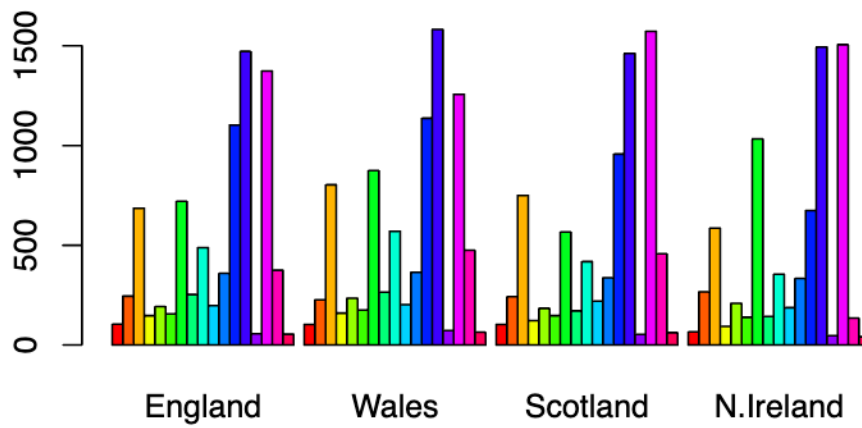
Q2: Which approach to solving the 'row-names problem' mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

Using `row.names=1` directly in `read.csv()` is better. The manual `x <- x[,-1]` approach will keep deleting the first column every time you re-run the chunk, which can cause bugs and is harder to catch.

Spotting Major Differences and Trends

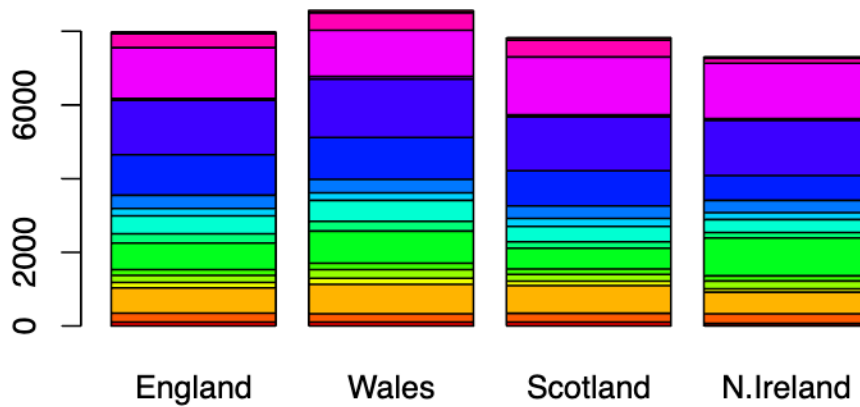
Notes: - Raw barplots with many categories get cluttered fast — hard to compare countries - `beside=TRUE` puts bars side by side; removing it gives a stacked barplot - These plots aren't very useful here — this motivates why we need PCA

```
barplot(as.matrix(x), beside=TRUE, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above `barplot()` function results in the following stacked plot?

```
barplot(as.matrix(x), beside=FALSE, col=rainbow(nrow(x)))
```



Changing `beside=TRUE` to `beside=FALSE` produces a stacked barplot.

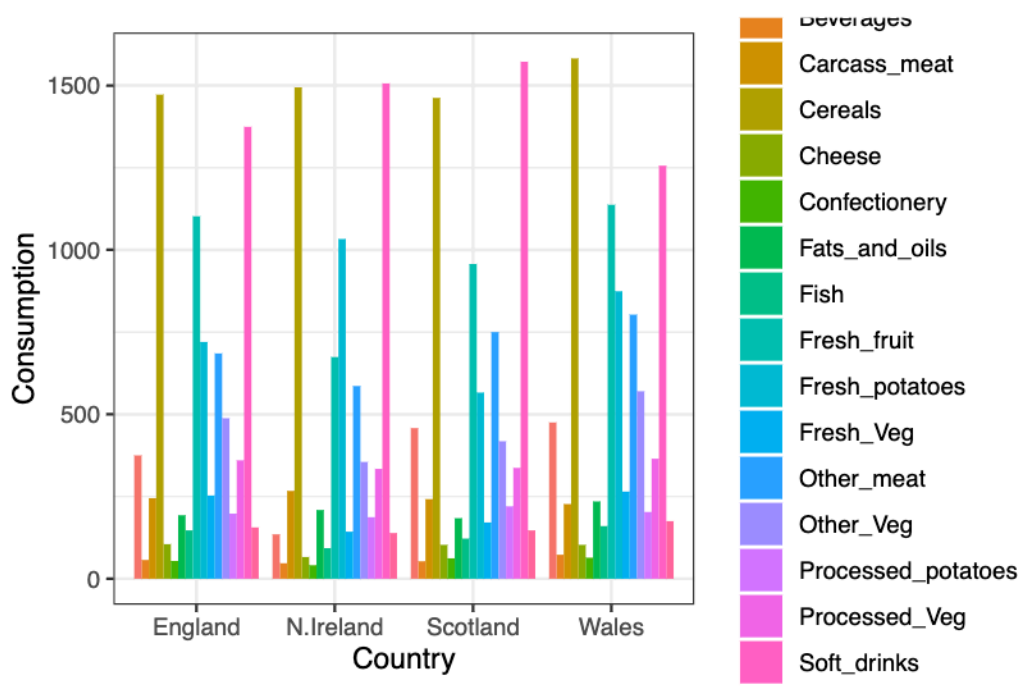
ggplot Version (tidy data)

Notes: - ggplot requires **long format** data — one row per measurement - `pivot_longer()` from `tidyr` converts wide to long - Wide: 17 rows $\times$ 4 columns; Long: 68 rows $\times$ 3 columns (17 foods $\times$ 4 countries)

```
library(tidyr)
library(ggplot2)

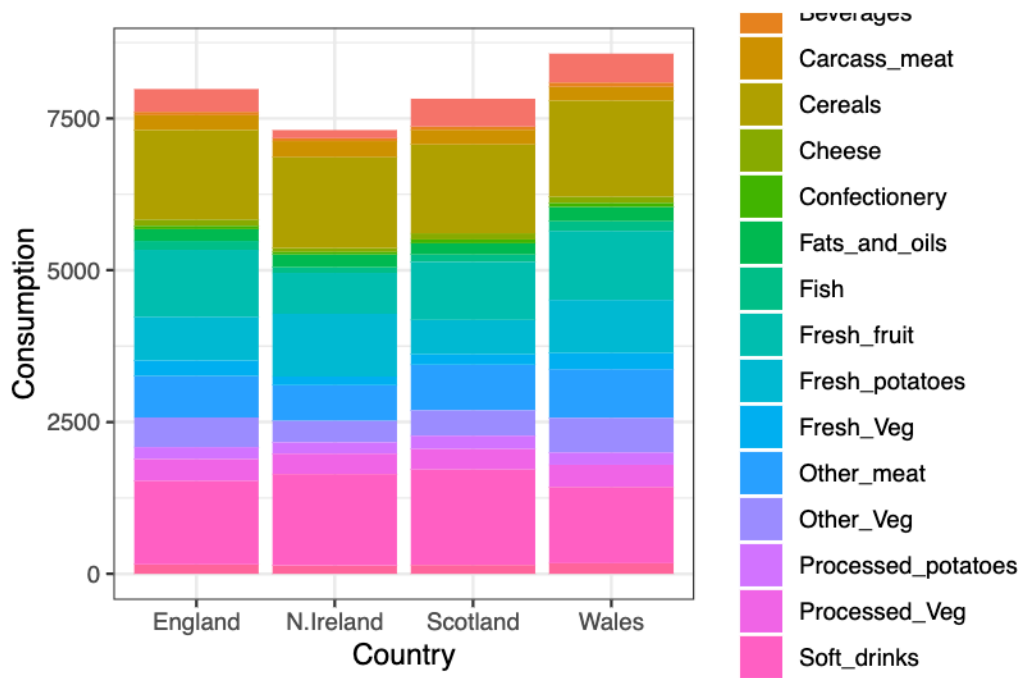
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4: Changing what optional argument in the above `ggplot()` code results in a stacked barplot?

```
ggplot(x_long) +  
  aes(x = Country, y = Consumption, fill = Food) +  
  geom_col(position = "stack") +  
  theme_bw()
```

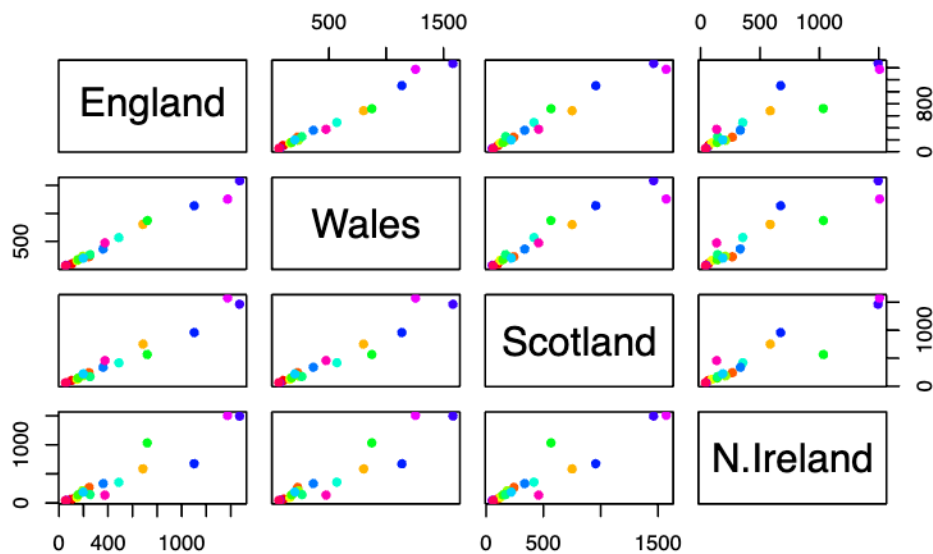


Changing `position = "dodge"` to `position = "stack"` gives a stacked barplot.

Pairs Plots and Heatmaps

Notes: - `pairs()` shows every pairwise scatterplot between countries simultaneously - Points on the diagonal = that food is consumed in equal amounts in both countries - Points off the diagonal = differences in consumption between those two countries - Heatmaps use color to represent values — easier to spot patterns at a glance - The dendrograms on a heatmap show hierarchical clustering of rows and columns

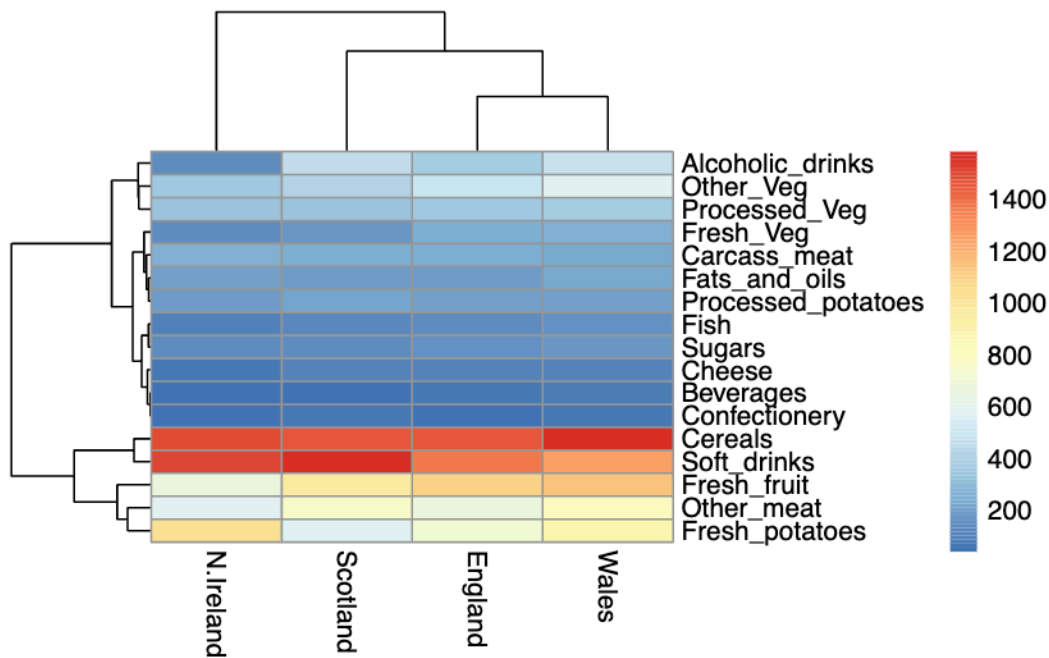
```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



Q5: Can you make sense of the `pairs()` code and figure? What does it mean if a given point lies on the diagonal for a given plot?

Each panel in the pairs plot is a scatterplot comparing food consumption between two countries. Each of the 17 points represents one food type. If a point lies on the diagonal it means both countries consume that food in the same amount — no difference between them for that food. Points that fall far off the diagonal are foods where the two countries differ most. For example N. Ireland vs England shows several points far off the diagonal, particularly for fresh potatoes.

```
library(pheatmap)
pheatmap(as.matrix(x))
```

Q6: Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns?

England, Wales, and Scotland all cluster together on the heatmap dendrogram, meaning their food consumption patterns are very similar to each other. N. Ireland clusters separately, driven by notably higher fresh potato and soft drink consumption and lower fresh fruit and alcohol consumption. This pattern is visible in the pairs plot too but much easier to see in the heatmap.

PCA to the Rescue

Notes: - `prcomp()` is R's built-in PCA function - **Must transpose** with `t()` — `prcomp()` expects samples as rows, variables as columns - Here our samples are countries and our variables are foods - `scale=TRUE` matters when variables are on different scales — not critical here since all values are in grams

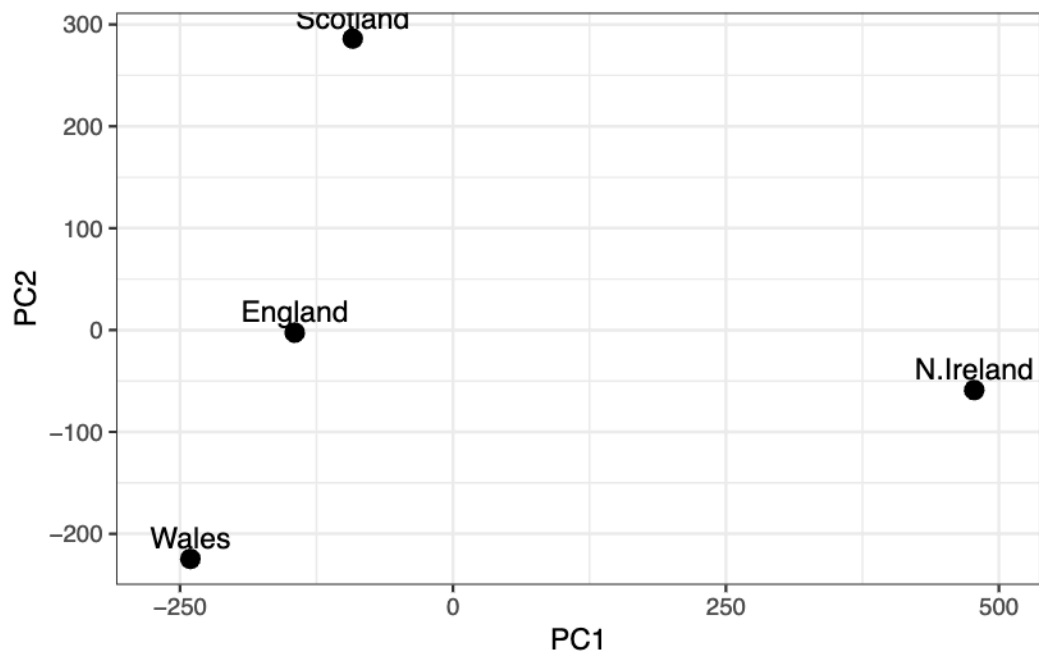
```
pca <- prcomp(t(x))
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

Q7: Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

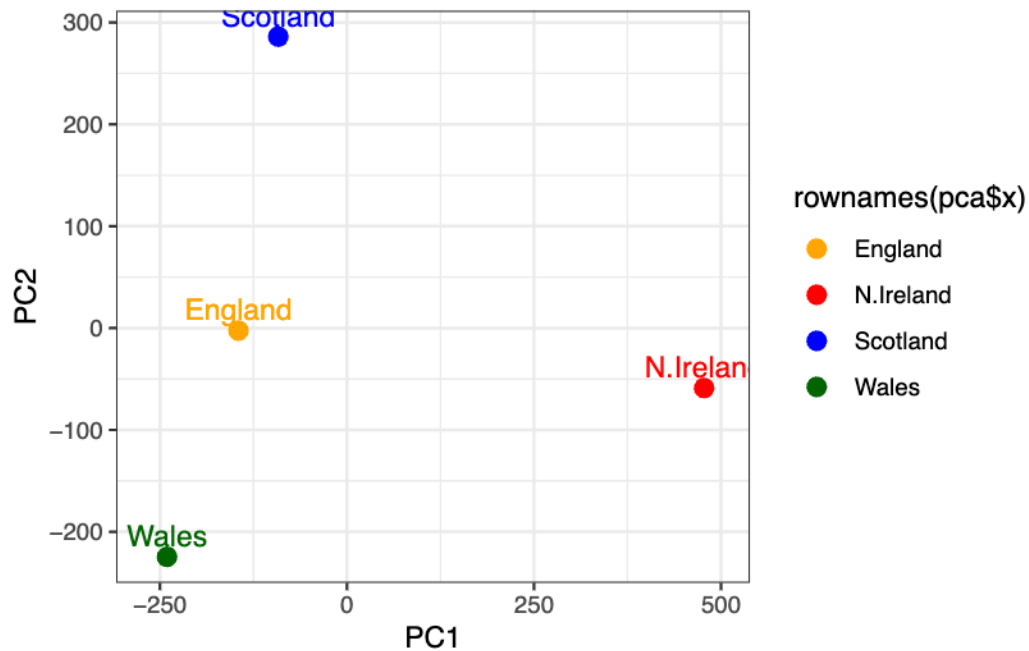
```
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Q8: Customize your plot so that the colors of the country names match the colors in the UK and Ireland map at the start of this document.

```
country_colors <- c("orange", "red", "blue", "darkgreen")

ggplot(as.data.frame(pca$x)) +
  aes(x = PC1, y = PC2, label = rownames(pca$x), color = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, show.legend = FALSE) +
  scale_color_manual(values = country_colors) +
  xlim(-270, 500) +
  xlab("PC1") + ylab("PC2") +
  theme_bw()
```



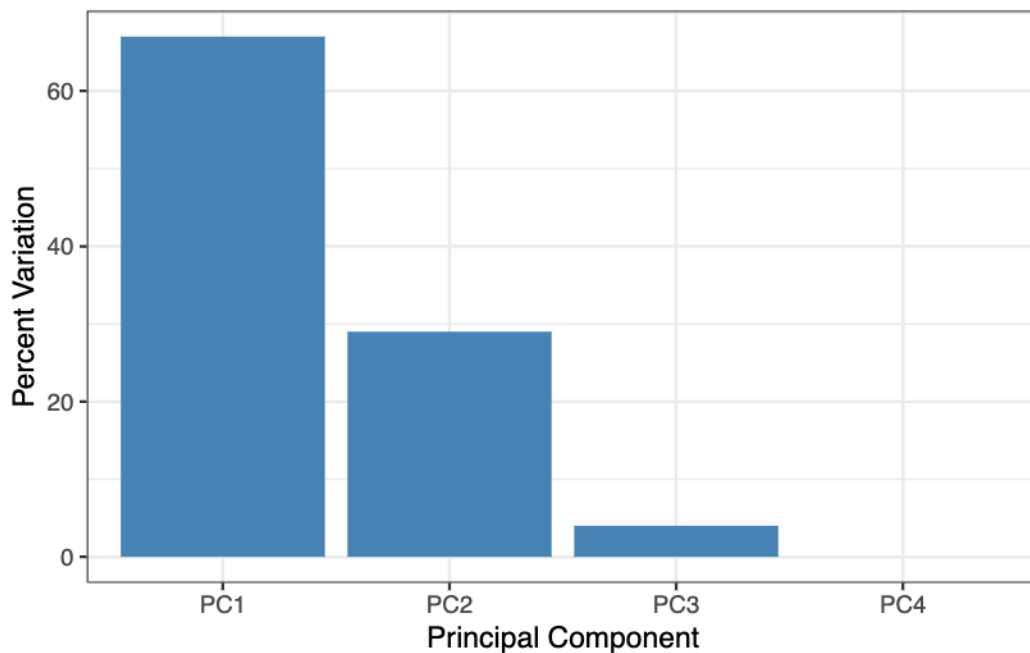
Scree Plot

Notes: - Scree plot shows how much variance each PC captures - Look for the “elbow” — diminishing returns after that point - Here PC1 + PC2 = ~97% of total variance so the 2D plot is an excellent summary of the data

```
v <- round(pca$sdev^2 / sum(pca$sdev^2) * 100)

variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)

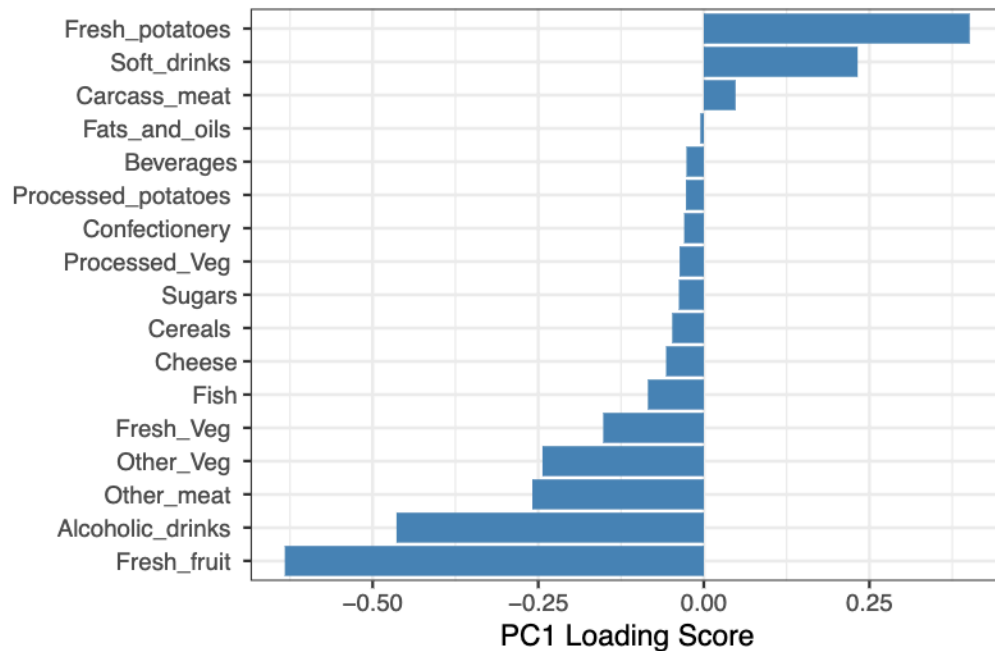
ggplot(variance_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw()
```



Digging Deeper — Variable Loadings

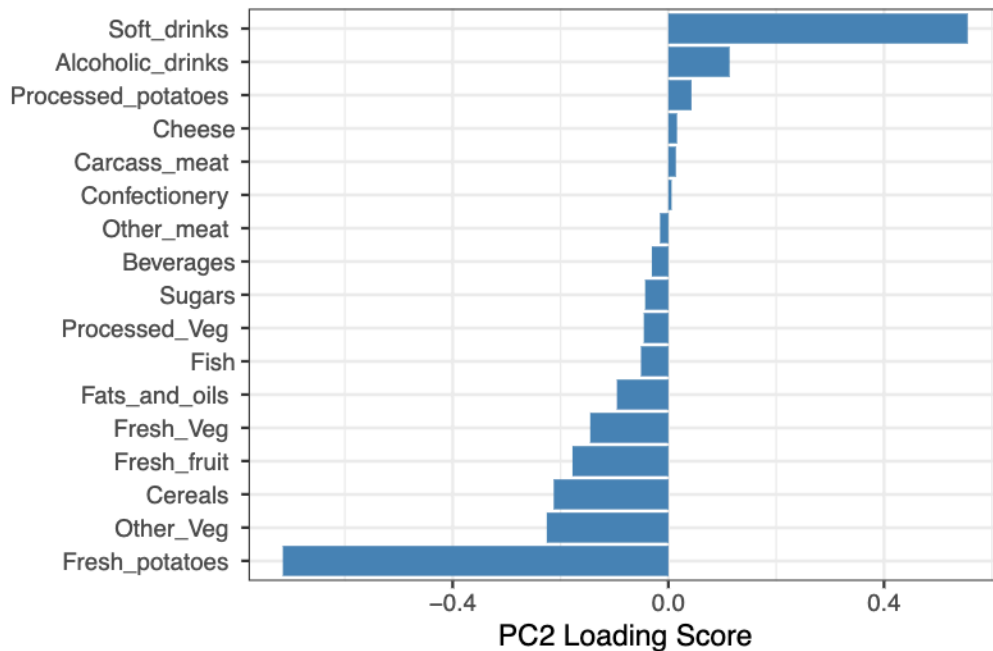
Notes: - Loadings tell you which foods drive each PC - High positive loading → that food pushes a sample to the right on PC1 (toward N. Ireland) - High negative loading → pushes samples to the left (toward England/Wales/Scotland) - This is how you interpret what the PCA is actually capturing biologically

```
ggplot(pca$rotation) +  
  aes(x = PC1, y = reorder(rownames(pca$rotation), PC1)) +  
  geom_col(fill = "steelblue") +  
  xlab("PC1 Loading Score") +  
  ylab("") +  
  theme_bw() +  
  theme(axis.text.y = element_text(size = 9))
```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```
ggplot(pca$rotation) +
  aes(x = PC2, y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Fresh potatoes (positive) and soft drinks (negative) are the two most prominent foods on PC2. PC2 mainly captures the difference between Scotland (high soft drink consumption) and Wales (high fresh potato consumption).

Section 2: PCA of RNA-seq Data (Optional)

Background

Notes: - PCA is extremely common in RNA-seq workflows for QC and exploratory analysis - If wt and ko samples don't separate cleanly that would be a red flag - `scale=TRUE` is especially important here since gene expression values vary wildly in magnitude across genes

```
url2 <- "https://tinyurl.com/expression-CSV"
rna.data <- read.csv(url2, row.names=1)
head(rna.data)
```

	wt1	wt2	wt3	wt4	wt5	ko1	ko2	ko3	ko4	ko5
gene1	439	458	408	429	420	90	88	86	90	93
gene2	219	200	204	210	187	427	423	434	433	426

```
gene3 1006 989 1030 1017 973 252 237 238 226 210
gene4  783 792  829  856 760 849 856 835 885 894
gene5  181 249  204  244 225 277 305 272 270 279
gene6  460 502  491  491 493 612 594 577 618 638
```

Q10: How many genes and samples are in this dataset? How many PCs do you think it will take to have a useful overview?

```
dim(rna.data)
```

```
[1] 100  10
```

100 genes (rows) and 10 samples (columns). Since there are only 2 conditions (wt vs ko), I'd expect just PC1 to capture most of the variation — and it does, at 92.6%.

```
pca <- prcomp(t(rna.data), scale=TRUE)
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	9.6237	1.5198	1.05787	1.05203	0.88062	0.82545	0.80111
Proportion of Variance	0.9262	0.0231	0.01119	0.01107	0.00775	0.00681	0.00642
Cumulative Proportion	0.9262	0.9493	0.96045	0.97152	0.97928	0.98609	0.99251

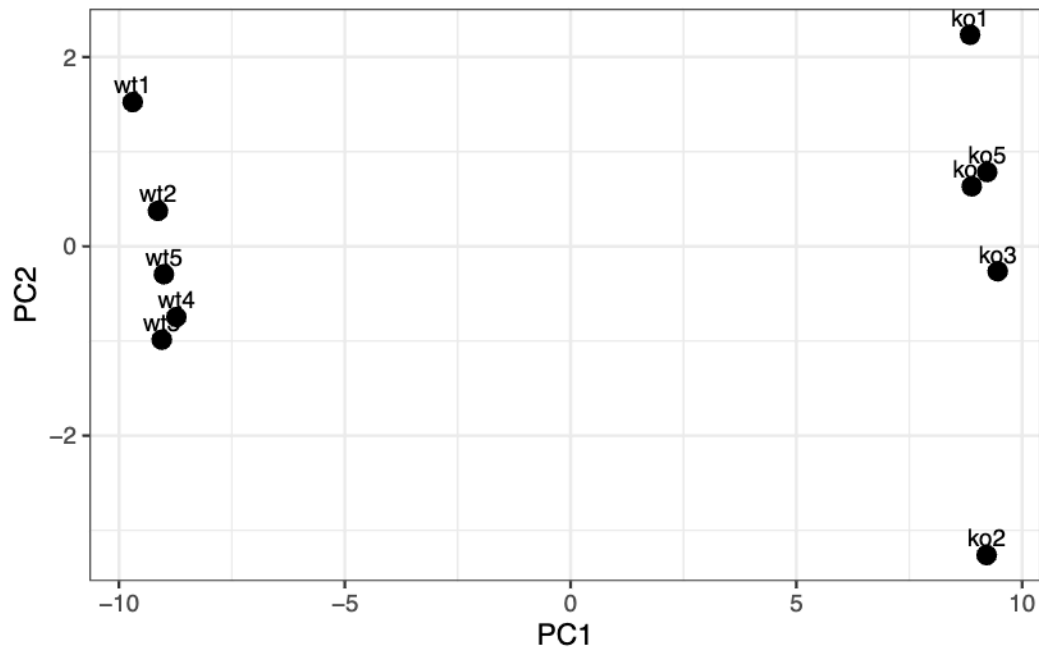
	PC8	PC9	PC10
Standard deviation	0.62065	0.60342	3.3e-15
Proportion of Variance	0.00385	0.00364	0.0e+00
Cumulative Proportion	0.99636	1.00000	1.0e+00

```
df <- as.data.frame(pca$x)
df$Sample <- rownames(df)

ggplot(df) +
  aes(x = PC1, y = PC2, label = Sample) +
  geom_point(size = 3) +
```



```
geom_text(vjust = -0.5, size = 3) +
xlab("PC1") + ylab("PC2") +
theme_bw()
```

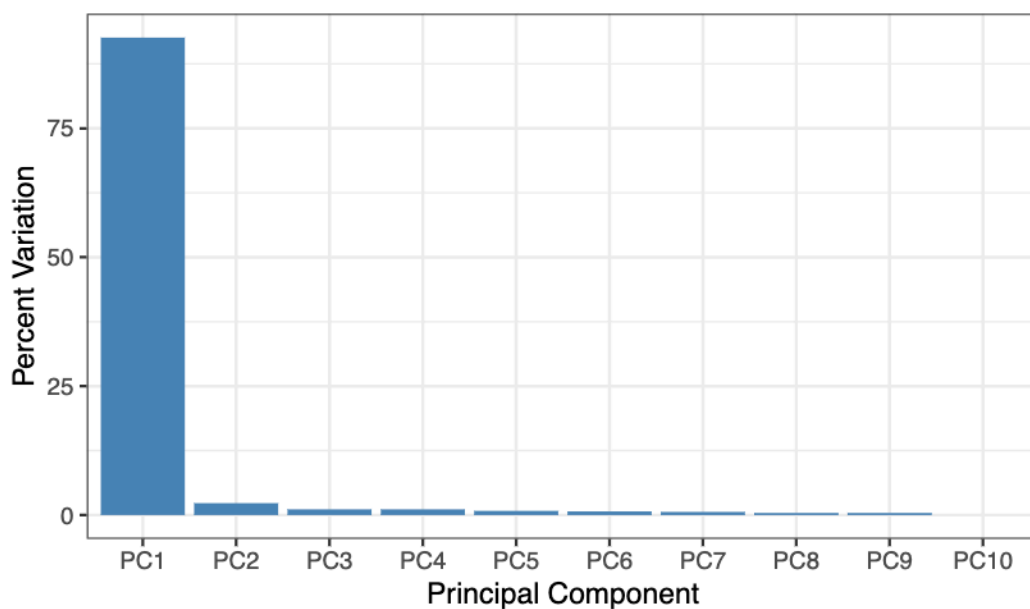


```
pca.var <- pca$sdev^2
pca.var.per <- round(pca.var / sum(pca.var) * 100, 1)

screedf <- data.frame(
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),
  PercentVariation = pca.var.per[1:10]
)

ggplot(screedf) +
  aes(x = PC, y = PercentVariation) +
  geom_col(fill = "steelblue") +
  ggtitle("Scree Plot") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw()
```

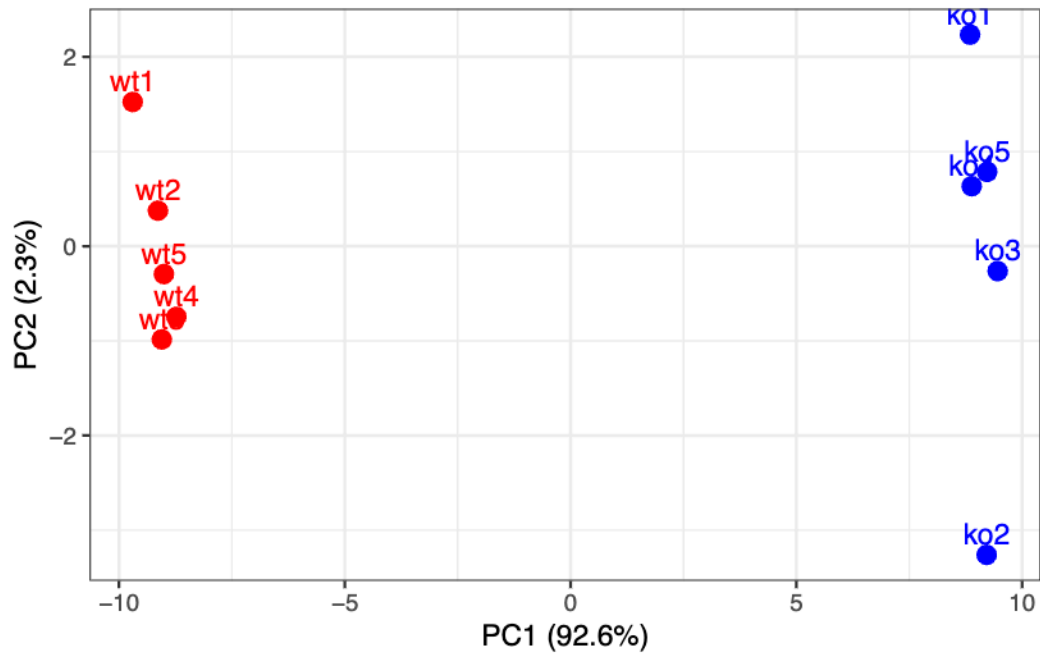
Scree Plot



```
colvec <- colnames(rna.data)
colvec[grep("wt", colvec)] <- "red"
colvec[grep("ko", colvec)] <- "blue"

df$color <- colvec

ggplot(df) +
  aes(x = PC1, y = PC2, color = color, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, hjust = 0.5, show.legend = FALSE) +
  scale_color_identity() +
  xlab(paste0("PC1 (", pca.var.per[1], "%)")) +
  ylab(paste0("PC2 (", pca.var.per[2], "%)")) +
  theme_bw()
```



```
loading_scores <- pca$rotation[,1]
gene_scores <- abs(loading_scores)
gene_score_ranked <- sort(gene_scores, decreasing=TRUE)
top_10_genes <- names(gene_score_ranked[1:10])
top_10_genes
```

```
[1] "gene100" "gene66" "gene45" "gene68" "gene98" "gene60" "gene21"
[8] "gene56" "gene10" "gene90"
```
